Towards Robust Cache Hits with Scalability via a Middle Layer Representing Workload Preferences

Kunpeng Ding[†] dingkp@mail.ustc.edu.cn
Jiangping Han[†*] jphan@ustc.edu.cn
Kaiping Xue[†*] kpxue@ustc.edu.cn
Qiangqiang Wei[†] weiqiangqiang@mail.ustc.edu.cn
[†] University of Science and Technology of China, Hefei Anhui, China

Abstract

Cache effectiveness heavily depends on the eviction policy. Learning-based eviction policies have improved over heuristics by adapting to workload characteristics, but most existing designs still rely on static object utilities as learning targets, whose semantics remain fixed across workloads and may deviate from the optimal eviction objective, especially when their assumptions do not match the workload. Moreover, utility prediction is often placed on the critical eviction path, forcing the cache to wait for model inference before selecting a victim. This serial dependency raises scalability concerns under high-rate request arrivals.

To address these limitations, we propose RSM, which achieves robust cache hits and scalable request processing via a workload-preference middle layer. Rather than learning a static object utility, our model learns how the current workload prioritizes different eviction-relevant features. We express this preference as a parameterized scoring formula over a concise set of representative features at the middle layer. The formula is simple enough to execute on every eviction, allowing the cache to select victims directly on the critical path without per-eviction model inference. The learned model is moved off the critical path and updates the middle layer intermittently to reflect the latest workload preferences, preserving eviction quality while improving scalability. Extensive simulations across 5,768 real-world traces show that RSM consistently outperforms state-of-the-art eviction policies in miss ratio without sacrificing heuristic-level throughput, and a Cachelib prototype confirms its practicality in production caches.

CCS Concepts: • **Theory of computation → Caching and paging algorithms**; • **Information systems → Information storage systems**.

Keywords: Cache eviction policy, middle layer, workload preference

*corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SIGOPS ATC 2026, Hong Kong, China

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2818-1/2026/11
https://doi.org/10.1145/3828523.3857083

ACM Reference Format:
Kunpeng Ding, Jiangping Han, Kaiping Xue, and Qiangqiang Wei. 2026. Towards Robust Cache Hits with Scalability via a Middle Layer Representing Workload Preferences. In *The 2026 ACM SIGOPS Annual Technical Conference (SIGOPS ATC 2026), November 15–18, 2026, Hong Kong, China.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3828523.3857083

1 Introduction

The effectiveness of a caching system heavily relies on its eviction policy. Traditional heuristic-based policies [5, 14] are lightweight and fast, but they are often highly environment-specific and can degrade when workload patterns shift. To improve adaptability, learning-based policies [17, 40] have attracted increasing attention for their ability to fit richer workload patterns and achieve lower cache miss ratios. They are commonly built around object utilities such as popularity, hit probability, and time-to-next-request of objects. Specifically, a learned model predicts the utility of cached objects, and the cache evicts objects with the weakest predicted utility. However, object-utility learning leaves two limitations in production caches:

Limitation 1: learning target. Although the learned model adapts to workload observations, the utility target's semantics remain fixed. When workload characteristics change, the assumptions behind the target may no longer hold, creating a mismatch with the eviction objective. Learning to predict such utilities can inherit this mismatch, and prediction errors can further amplify suboptimal evictions (§ 2.1).

Limitation 2: eviction paradigm. Object-utility learning places model inference on the critical eviction path. Before selecting a victim, the cache must first predict utilities for many candidate objects. This serial dependency places model inference inside request processing and raises scalability concerns under high-rate request arrivals (§ 2.2).

These two limitations call for rethinking both what to learn and how to use the learned model during eviction. The learning target should move beyond static object utilities and instead capture workload-adaptive eviction preferences. The eviction paradigm should also decouple model inference from victim selection: the model must not be invoked before every eviction, while a lightweight structure should guide fast decisions on the critical path.

To begin with, we reshape the eviction paradigm by introducing a middle layer between model inference and eviction decisions, thereby decoupling them. The middle layer directly guides each eviction decision, while the model is invoked only intermittently to update the middle-layer parameters. This new paradigm allows the cache to make real-time eviction decisions without waiting for slow model inference, thus maintaining low latency under high-rate request arrivals. Meanwhile, the middle layer replaces the object utility as the learning target and is updated intermittently by the model to preserve eviction quality.

The middle layer is central to this new eviction paradigm, as it defines both the critical-path eviction logic and the off-path learning target. It must meet three requirements:
(R1) **Simplicity**: it should incur minimal computational overhead to ensure fast, real-time eviction decisions while remaining easily optimizable by learning models;
(R2) **Universality**: it should be expressive enough to capture and represent a wide variety of workload patterns;
(R3) **Stability**: the learned parameters should remain effective between model updates.

The middle layer design is inspired by the common principle behind existing policies (§ 3). Both heuristic and learning-based policies implicitly or explicitly rely on features that are considered informative for eviction decisions. They either directly use features, or derive a metric from raw features, or learn a utility from raw features, which we collectively call eviction signals. However, the semantics of the eviction signal used to rank victims are fixed. Once the signal does not match the current workload, the eviction decisions relying on it can degrade. We therefore move from fixed to dynamic feature use. Rather than only contributing to a static eviction signal, features directly guide eviction decisions, with their emphasis adjusted according to observed performance under changing workloads. RSM realizes this dynamic use through adjustable feature weights in a parameterized scoring formula. In fact, the weights provide a bridge from feature importance to workload-level eviction preference.

This formulation meets the above requirements. First, the formula only involves simple arithmetic operations, keeping execution fast, and contains only a small number of parameters, making it easy to learn ((R1)). Second, it allows multiple informative signals to jointly participate in eviction decisions with adjustable importance, so that different weight combinations can represent different workload preferences ((R2)). Third, the learned workload-level feature importance usually changes slowly, allowing the same middle-layer parameters to remain effective across many eviction decisions within a temporal window ((R3)).

Built around this, we propose RSM, a cache eviction framework that achieves **R**obust cache hits and **S**calable request processing through a workload-preference **M**iddle layer. The system consists of a decision layer for cache eviction decisions and a model layer for learning and inference. The decision layer must be equally lightweight to preserve the low-latency advantage of the scoring formula. To achieve this, we introduce an efficient candidate selection mechanism, eliminating the need to score all cached objects during an eviction. We also design a score-reuse eviction strategy that reuses previously computed scores when they remain reliable. The model layer runs intermittently outside the latency-critical eviction path, learning the middle-layer weights from cache feedback. We use the Covariance Matrix Adaptation Evolution Strategy (CMA-ES), which searches this low-dimensional parameter space without requiring gradients, while its covariance adaptation captures feature interactions and its restart mechanism improves robustness under shifting workloads.

Our contributions are summarized as follows:

- We propose a scalable eviction paradigm that changes how the learned model participates in cache eviction. Instead of invoking the model before each eviction, RSM decouples them by using a middle layer to directly guide eviction decisions and lets the model update this layer only intermittently, enabling low-latency decisions under high-rate request arrivals.
- We design a workload-preference learning target for robust cache hits. Instead of learning a static object utility, RSM learns workload-level dynamic eviction preferences and encodes them as the weights of critical features in a parameterized scoring formula, which is highly expressive to model diverse workloads and provides a compact optimization target for CMA-ES.
- We extensively evaluate RSM using 5,768 real-world traces. The results demonstrate that our approach consistently outperforms eleven state-of-the-art baselines in cache miss ratios, while maintaining throughput comparable to heuristic policies. We further implement the prototype and confirm the practicality of RSM in production systems.

2 Motivation

Effective learning-based cache eviction policies commonly follow an object-utility learning paradigm. They first define a utility representing the eviction priority of an object, train a model to predict that utility, and evict the object with the lowest predicted utility. Although this paradigm is intuitive and effective in many settings, it raises two fundamental questions for production caches. First, *what should the model learn*? Because the optimal eviction choice is difficult to express as a direct per-object training target, existing methods usually learn a static proxy utility. However, such a proxy utility may not be aligned with the best eviction decision. The learned policy therefore inherits this target

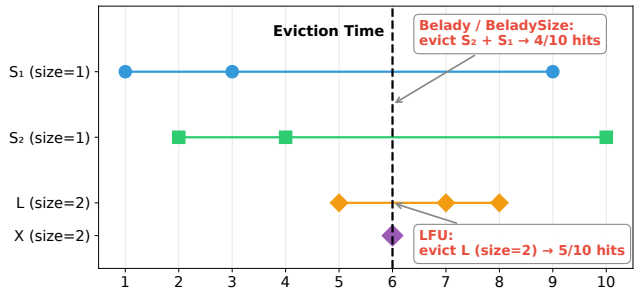


Figure 1. A short request sequence where Belady and BeladySize are suboptimal.

mismatch. Second, *how to use the learned model during eviction*? If model inference is invoked before each eviction, the cache must wait for the model to score objects before selecting victims, introducing substantial eviction latency.

2.1 Limitations of Object-Utility Learning Target

Learning-based policies define the object utility using eviction indicators with fixed semantics, including popularity, hit probability, and time-to-next-request. Among them, time-to-next-request is particularly influential [17, 40, 56], as it follows the logic of Belady's MIN algorithm (Belady for short) [6], the offline and proven optimal oracle algorithm for equal-size objects with complete knowledge of future requests. When all cached objects have the same size, evicting the object whose next request is farthest in the future is optimal. However, this target no longer remains optimal when object sizes vary [9], which is common in production caches. In this setting, the best eviction choice also depends on other factors such as how much cache space an object occupies. Still, BeladySize, which multiplies time-to-next-request by object size, can produce suboptimal decisions.

Fig. 1 illustrates this limitation with a short trace. The cache capacity is 4. Objects S_1 and S_2 have size 1, while objects L and X have size 2. On the request sequence $S_1, S_2, S_1, S_2, L, X, L, L, S_1, S_2$, the request for X requires freeing two units of space from a cache containing S_1, S_2, and L. Belady keeps L because its next request is the closest and evicts both small objects. BeladySize makes the same decision. This yields 4 hits in the 10-request trace. In contrast, Least Frequently Used (LFU) evicts L and keeps the two repeatedly requested small objects.

This example illustrates a potential mismatch between a static utility target and the eviction objective. If even an oracle with perfect knowledge of object utility produces a suboptimal decision, then a learning-based policy targeting this utility can inherit this gap even before accounting for prediction errors, delayed labels, and changing workloads. This motivates us to look beyond a static utility and examine what information actually provides useful eviction guidance in production traces and how to use it for eviction decisions.

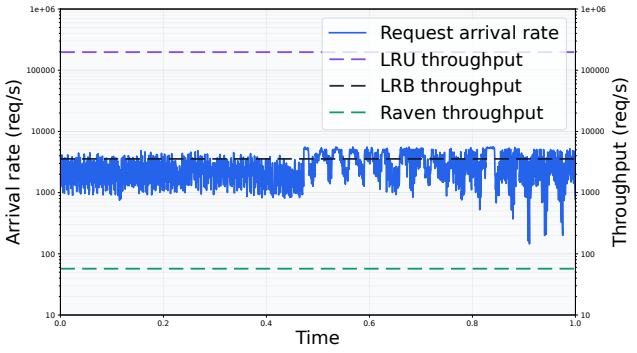


Figure 2. Throughput of representative learning policies and LRU on an Alibaba trace.

2.2 Limitations of Object-Utility Eviction Paradigm

Object-utility learning policies often make the selection of evicted victims conditional on model inference. The serial dependency in this eviction paradigm places model inference directly inside request processing and increases eviction latency inevitably.

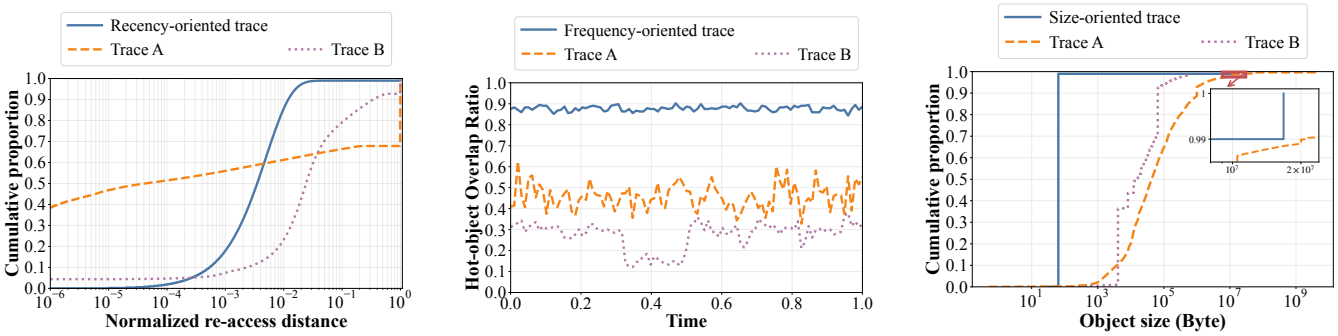
Since most eviction policies require meta-data updates under locking when executing read operations and thus cannot effectively increase throughput by adding more CPUs [52], this in-path model invocation raises a scalability concern because production caches have to sustain high-rate request arrivals [41]. In modern infrastructures, caches can receive tens of thousands of requests per second [50]. If each eviction spends comparable time on model inference, requests begin to queue and the cache can no longer keep up with the incoming load.

To quantify this latency gap in a realistic setting, we implement the prototypes of two representative learning-based policies LRB [40] and Raven [17] in Cachelib [7], and measure their throughput on an Alibaba trace. As shown in Fig. 2, LRB's throughput falls below the peak arrival rate of the trace, and Raven's throughput falls even below the average arrival rate. This measurement shows that heavy inference during eviction can consume the latency budget required by high-rate request arrivals.

Therefore, model inference should not be a prerequisite for every eviction. We introduce a middle layer to decouple model inference from eviction decisions. The middle layer acts as a lightweight eviction guider that participates in each eviction and preserves low-overhead decision making, while it is intermittently updated by the model.

3 Key Insights for the Middle Layer Design

The middle layer is designed to pursue expressiveness across diverse workloads. We are inspired by successful practices of existing eviction policies. Whether heuristic or learning-based, most policies implicitly or explicitly rely on one or a few critical features that are considered sufficiently informative for eviction decisions. For example, LRU relies



(a) Recency distribution (b) Frequency stability (c) Size distribution

Figure 3. Trace characteristics for recency, frequency, and size. Each panel compares the corresponding oriented synthetic trace with Trace A and Trace B.

on recency, LHD [5] relies on the expected cache hit count per unit size, and LRB [40] relies on the time-to-next-request. This principle suggests that critical features can guide eviction decisions. However, using these features in a fixed manner limits the performance since the importance of each feature varies in different workloads.

As an example, we examine three basic features widely used in cache eviction: recency, frequency, and object size. To examine the effectiveness of each feature, we associate each of them with simple eviction policies: Least Recently Used (LRU) that evicts the object whose latest access is farthest in the past, Least Frequently Used (LFU) that evicts the object with the fewest past requests, and Largest Size Used (LSU) that evicts the largest objects.

To isolate the role of each feature, we use three controlled synthetic traces and two production traces. Each synthetic trace deliberately makes one feature able to dominate eviction decisions. In the recency-oriented trace, requests concentrate on a small hot set that moves over time, so objects are often requested again after a short interval, making the trace LRU-friendly. In the frequency-oriented trace, requests follow a stable Zipf-like popularity distribution, so the same popular objects remain frequent across time windows, making the trace LFU-friendly. In the size-oriented trace, object sizes follow a two-tier distribution, where most objects belong to a small-size tier and only a small fraction belong to a clearly separated large-size tier, making the trace LSU-friendly. The production traces used are from Meta CDN and Tencent CBS, called Trace A and Trace B, respectively. Next we analyze each feature in these traces and evaluate how the corresponding feature-guided heuristic performs.

Recency feature. Fig. 3(a) plots the cumulative distribution of normalized re-access distance, which measures the interval between repeated requests for the same object. The recency-oriented trace places more probability mass at short re-access distances, and has almost no one-hit wonders

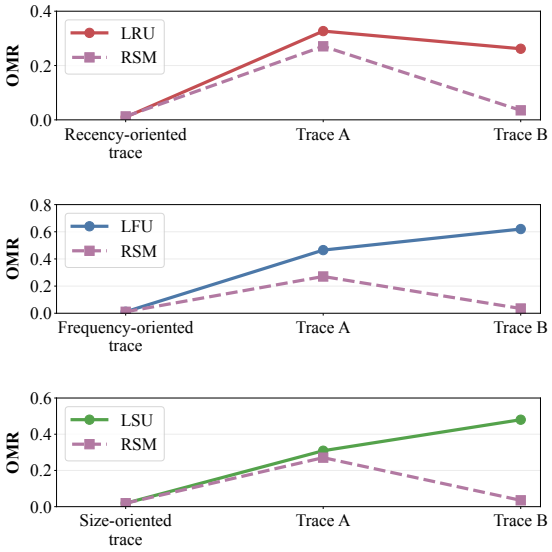


Figure 4. Object miss ratios of LRU, LFU, LSU, and RSM on each oriented synthetic trace, Trace A, and Trace B.

(occurring only one time), reflected by the right edge of the figure. In contrast, the production traces show a visible one-hit wonder ratio. This difference is reflected in the top panel of Fig. 4. LRU achieves a miss ratio of 0.01 on the recency-oriented trace, but increases to 0.33 on Trace A and 0.26 on Trace B. The miss ratio gap shows that recency is a reliable eviction feature when short re-access is consistently strong.

Frequency feature. Fig. 3(b) plots the hot-object overlap ratio between adjacent time windows, which captures whether currently popular objects remain popular in the near future. The frequency-oriented trace maintains a high overlap ratio. Trace A has lower overlap, and Trace B shows a further decrease, indicating that historical popularity changes quickly and becomes less predictive. The middle panel of Fig. 4 shows the corresponding miss ratio for LFU.

LFU achieves 0.01 on the frequency-oriented trace, but 0.47 on Trace A and 0.62 on Trace B. The miss ratio gap shows that frequency is useful when popularity is sufficiently stable.

Size feature. Fig. 3(c) plots the cumulative distribution of object sizes. In the size-oriented trace, the early rise corresponds to the small-object tier, while the highlighted tail region marks the later rise for the large-object tier. This separation makes size a clear eviction signal, because evicting objects from the large tier can free space for multiple smaller objects. Trace A has a more dispersed size distribution than Trace B, suggesting that size is more informative on Trace A. The bottom panel of Fig. 4 shows the corresponding miss ratio for LSU. LSU reaches 0.02 on the size-oriented trace, but rises to 0.31 on Trace A and further to 0.48 on Trace B. The miss ratio gap shows that size is effective when object sizes are sufficiently dispersed.

These comparisons reveal that some critical features have potential value for eviction, since the corresponding heuristic performs well on the synthetic trace where that feature is deliberately strengthened. Moreover, the strength of each feature varies across production traces. When a feature becomes weak, unstable, or mixed with other features, the policy that relies on it degrades. This suggests that it is promising to learn how much each feature should be emphasized under the current workload. Following this idea, RSM represents such emphasis through the weights of a scoring formula. As shown in Fig. 4, RSM remains close to the matching single-feature specialist on the three synthetic traces and achieves the lowest miss ratio on both production traces, with 0.27 on Trace A and 0.04 on Trace B.

4 Design of RSM

4.1 Overview

Fig. 5 illustrates the proposed RSM, which is organized into three layers: a *decision layer* for making eviction decisions, a *middle layer* for guiding the decision layer, and a *model layer* for updating the middle layer.

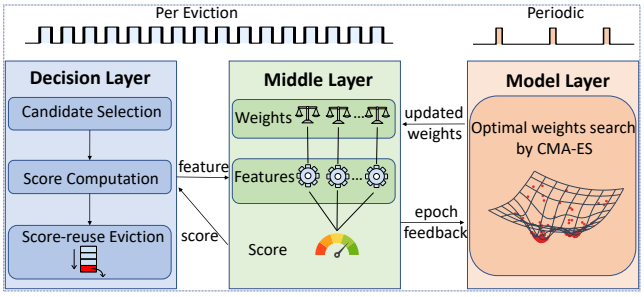


Figure 5. Overview of the proposed RSM.

The core of RSM is the middle layer, which ensures low-latency eviction decisions by removing their prerequisite dependency on model inference, and provides the potential for robust cache hits by representing workload-level eviction preferences as the learning target. Concretely, it is formed as a parametric scoring formula consisting of critical features with adjustable weights. The middle layer uses this formula as its interface with the other two layers: it guides the decision layer during eviction and receives weight updates from the model layer. The decision layer applies the workload preference represented by the middle layer to each eviction. It first selects a small set of likely low-value candidates, computes their scores using the middle-layer formula, and finally evicts the lowest-scoring object. The decision layer and the middle layer are lightweight and run on the per-eviction path. In contrast, the model layer runs periodically outside the per-eviction path, keeping the middle layer aligned with recent workload preferences. It observes cache statistics, searches for better weights with CMA-ES, and refreshes the middle-layer parameters.

4.2 Middle Layer Design

The middle layer is the core component through which RSM takes effect. It must satisfy three requirements: simplicity for per-eviction execution, universality across heterogeneous workloads, and stability between model updates.

These requirements lead to targeting workload-level eviction preferences represented by a parametric scoring formula. We select age, frequency, and size as the three primary descriptors because they are readily available, directly related to eviction quality, and characterize cached objects along different basic dimensions. Since these quantities can differ by several orders of magnitude, directly combining them would cause larger-scale features to dominate smaller ones and make the score unstable across workloads. The logarithmic transform preserves the original ordering and compresses heavy-tailed extremes so that a small number of extreme observations do not dominate the scoring formula. Therefore, before scoring, raw values x are first log-compressed, $\tilde{x} = \log(1 + x)$, and then normalized to $[0, 1]$ using the running minimum and maximum observed so far:

$$\hat{x} = \frac{\tilde{x} - \tilde{x}_{\min}}{\tilde{x}_{\max} - \tilde{x}_{\min}}, \tag{1}$$

which keeps the scales of different features comparable across workloads. The transformed age, frequency, and size are denoted by a, f, and s, respectively.

A plain linear combination of the three first-order terms is insufficient for production workloads, because real traces often involve cross-dimensional interactions. Although a neural network could model such interactions, our tests show its poor performance in this setting, likely due to its larger parameter space and harder convergence. We therefore use a small set of pairwise interaction terms to improve expressiveness while keeping the parameter space low-dimensional and easier to optimize. Among the three primary features, age is the most important because it directly captures current staleness and determines whether

popularity and size should still matter for retention. We accordingly add second-order terms centered on age: f/a measuring whether an object remains popular relative to its age, and as measuring how long the object's occupied space has remained unaccessed. This design keeps the online score as a small number of arithmetic operations, while leaving only a compact set of interpretable weights for the model layer to optimize. The resulting scoring formula is

$$S(o) = -w_1 a + w_2 f - w_3 s + w_4 \frac{f}{a} - w_5 as, \tag{2}$$

where each weight $w_i \in [0, 1]$ is periodically updated by the model layer. The signs are fixed by cache semantics. Large age and large size increase evictability, while large frequency increases retainability. Fixing the signs reduces the search space by letting the learner optimize only non-negative weight magnitudes.

Equation (2) subsumes a broad family of classical heuristics as special or soft cases. The first-order terms provide the basic eviction preferences: emphasizing only a approximates age-based eviction, emphasizing f approximates frequency-oriented retention, and emphasizing s favors size-aware replacement. The interaction terms then introduce cross-dimensional preferences. Specifically, f/a strengthens protection for objects whose popularity remains high relative to their age, while as suppresses stale large objects that are expensive to keep. By combining these first-order and interaction terms with learnable weights, the middle layer can represent richer multidimensional eviction preferences than any individual heuristic, while remaining efficient in the per-eviction path.

4.3 Decision Layer Design

The decision layer executes on every cache miss that requires replacement. Its objective is to preserve eviction quality while avoiding full-cache scanning. This is achieved through a three-step pipeline: candidate selection, score computation, and score-reuse eviction.

1) Candidate selection. In realistic deployments, a cache can contain thousands to tens of millions of objects. Even a lightweight scoring function becomes costly if it is repeatedly evaluated for all cached residents on every eviction. We therefore construct a compact candidate pool consisting of objects that are likely to yield the lowest eviction score.

To efficiently assemble this pool, we actively track objects that exhibit extreme values across the three fundamental dimensions: age, frequency, and size. The supporting structures are designed according to each feature's value range and update dynamics, so that both extraction and maintenance remain lightweight. For age, we maintain an age-ordered queue and sample objects directly from its tail to obtain the stalest residents. For frequency, we maintain count-indexed linked lists and sample from the lowest-count lists. For size, objects are grouped into discrete size-range buckets, where each bucket represents an interval of sizes rather than one exact size, and candidates are sampled from the buckets covering the largest ranges. The maintenance overhead of these structures is low: each request triggers only local insertion or movement of the touched object in the corresponding queue, list, or bucket, without global reordering or full-cache traversal. Candidates systematically drawn from these structures are called *feature-extreme candidates*, as they represent the worst objects along at least one individual dimension.

The quota assigned for feature-extreme candidates is not fixed across the three sources. Instead, we adaptively reallocate quota according to how often each source contributes the final eviction object. Let n_i^{evict} denote the number of evictions whose victim was selected from source i in the last period. The next-period quota for source i is adjusted in proportion to its observed contribution:

$$n_i^{\text{cand}} \propto n_i^{\text{evict}}. \tag{3}$$

A minimum quota is still enforced for each source to prevent starvation. The above adaptive mechanism directs more search effort to sources that are currently more predictive, while still preserving diversity in candidate generation.

Feature-extreme candidates alone are insufficient because the truly optimal eviction target may not be strictly extreme in any single feature. An object can be only moderately poor along each individual dimension, yet become the worst target under joint scoring. To cover such cases, we also sample a set of *random candidates* from current cached objects. Random candidates are sampled uniformly from cache residents. This random set complements the candidate pool, improving exploration over the full candidate space.

2) Score computation. For each candidate in the pool, we evaluate age, frequency, and size using the log-compression and running min–max normalization pipeline defined in Eq. (1). The raw age of an object is measured by the request distance since its last access, frequency is its access count, and size is its occupied bytes. The normalized features are then substituted into the middle-layer scoring formula (Eq. (2)) to produce an eviction priority for each candidate. Computing features and scores only for the selected candidates rather than all cached objects keeps per-eviction overhead inexpensive.

3) Score-reuse eviction. After score computation, candidates are ranked in ascending order of score. The lowest-scoring candidate is selected as the eviction victim, while the next several low-scoring candidates are retained in a score-ordered buffer. This buffer allows subsequent evictions to reuse the buffered ranking instead of repeating the full candidate collection and scoring procedure.

Whether the buffered ranking can be directly reused depends on what occurs between consecutive evictions. When a single large object admission triggers multiple sequential evictions within the same request, no other

accesses intervene, and all buffered candidates retain the features recorded at scoring time. The buffer therefore remains accurate, and its candidates can be evicted in the recorded order. When evictions span separate requests, however, some buffered objects may have been accessed in the meantime. Such accesses increase their frequency or recency, making them less suitable for eviction than their recorded scores indicate. The decision layer therefore checks each candidate's current state before eviction. If a candidate is inconsistent with its recorded state, it is skipped and the next buffered candidate is examined. If the buffer is exhausted, a fresh candidate collection and scoring procedure is performed.

The buffer further maintains its correctness through two mechanisms. First, when a new object enters the cache, it is inserted into the buffer if its score makes it more evictable than existing buffered candidates, ensuring that newly admitted low-value objects remain under consideration. In addition, when the model layer updates the weights, buffered scores become invalid. Therefore, the buffer is cleared and rebuilt at the next eviction.

Algorithm 1 summarizes the eviction procedure. At the start of each epoch, the decision layer updates the per-source quotas n_i^{cand} via Eq. (3), refreshes the weights, and clears the score-ordered buffer because buffered scores are no longer valid. During eviction, the decision layer first tries to reuse the buffer. If the buffer head exists and passes the state checks, it is selected as the eviction victim. Otherwise, a new candidate pool P is assembled from feature-extreme sources and random candidates. Each candidate is normalized according to Eq. (1) and scored according to Eq. (2). The lowest-scoring object is selected as the eviction victim, and the next several low-scoring candidates are retained in the buffer. After the missed object is admitted into the cache, it is inserted into the buffer if its score makes it more evictable than the existing buffered candidates.

Algorithm 1: Decision-layer eviction procedure.

Input: Missed request R, score-ordered buffer
Output: Evicted object(s)

if *at start of a new epoch* **then**
  update quota n_i^{cand} for each source i via Eq. (3);
  update weights and clear buffer;
end
while *free space < size(R)* **do**
  if *buffer nonempty (skip invalid heads)* **then**
    $O \leftarrow$ pop buffer head;
  else
    $P \leftarrow \emptyset$;
    add n_i^{cand} candidates to P from each source i;
    add random candidates to P;
    foreach $c \in P$ **do**
      normalize features per Eq. (1);
      compute score $S(c)$ per Eq. (2);
    end
    $O \leftarrow \arg\min_{c \in P} S(c)$;
    retain the next k candidates in buffer;
  end
  evict O and update statistics for source i of O;
end
if *the admitted R has the lowest score in buffer* **then**
  insert R into the buffer;
end

4.4 Model Layer Design

The model layer periodically updates the middle-layer weight vector $\mathbf{w} \in [0,1]^d$ to preserve its adaptability. Unlike learning-based policies that couple model inference with real-time eviction, RSM performs the heavier model computation only at epoch scale, separating slow optimization from the per-eviction decision path. It discovers workload eviction preferences and translates them into middle-layer weights, so the decision layer only needs to execute the resulting lightweight policy.

The model layer continuously executes a three-stage update loop: evaluating the current preference vector over an epoch, distilling a refined vector from the aggregated feedback, and updating the middle layer with the new policy. Let epoch t denote an evaluation window containing a sequence of requests. The model layer aims to minimize the epoch-level miss ratio (MR_t), which yields a low-dimensional black-box optimization over the middle-layer policy space:

$$\mathbf{w}_t^* = \arg \min_{\mathbf{w} \in [0,1]^d} \text{MR}_t(\mathbf{w}). \tag{4}$$

We solve this optimization problem using the Covariance Matrix Adaptation Evolution Strategy (CMA-ES), a derivative-free optimization algorithm for continuous spaces. It updates weights by repeated trial and selection, using miss-ratio feedback from completed epochs to refine the sampling distribution for future weight vectors.

This method is suitable for solving Eq. (4). Because the real cache environment is full of local optima and lacks reliable analytical gradients, direct mathematical optimization is difficult. Instead of relying on gradients, CMA-ES searches through sampled weight vectors and uses their miss-ratio feedback to refine the search distribution. By adapting a covariance matrix, it further captures second-order dependencies among feature dimensions.

Specifically, at each generation, CMA-ES maintains a sampling distribution over the middle-layer weights and draws a population of weight samples as

$$\mathbf{w}_k \sim \mathcal{N}(\mathbf{m}_t, \sigma_t^2 \mathbf{C}_t), \tag{5}$$

where $\mathbf{m}_t$ is the center of weight distribution, σ_t controls the overall search step, and $\mathbf{C}_t$ controls how different dimensions

Table 1. Summary of evaluation traces.

Dataset	Approx. time	# Traces	Cache type	# Requests (million)	# Objects (million)
Tencent Photo [55]	2018	2	Object	5,650	1,038
WikiMedia CDN [46]	2019	3	Object	2,863	56
Alibaba [26]	2020	1,000	Block	19,676	1,702
Tencent CBS [53]	2020	4,755	Block	33,690	551
Meta KV [32]	2022	5	Key-value	1,644	82
Meta CDN [32]	2025	3	Object	231	76

move together. The sampled vector is evaluated over an epoch to obtain its miss ratio $\mathrm{MR}_t(\mathbf{w}_k)$. Weight samples with lower miss ratios indicate more promising search directions for the next update. The center $\mathbf{m}_{t+1}$ is shifted toward these samples, while the covariance matrix $\mathbf{C}_{t+1}$ and step size σ_{t+1} are adjusted according to recent improvements. After the distribution is updated at the epoch boundary, a new weight vector sampled from the refined distribution is propagated to the middle layer, and subsequent evictions use the latest weights in the scoring formula.

To improve search robustness, we employ the aIPOP variant [4, 20] of CMA-ES. It actively accelerates convergence by suppressing unsuccessful search directions. Moreover, when the search stagnates or converges to a poor local optimum, it triggers a restart with an increased population size. This broadens exploration and helps the learner recover under multimodal and shifting real-world workloads.

5 Experimental Evaluation

We evaluate RSM to answer the following questions:

- How robust is RSM across heterogeneous workloads in terms of object miss ratio and byte miss ratio, compared with state-of-the-art policies? (§ 5.2)
- How scalable is RSM in terms of throughput, and does it remain comparable to heuristic policies? (§ 5.3)
- How practical is the RSM prototype? (§ 5.4)

5.1 Experimental Setup

Traces. Our evaluation is conducted on six open-source production datasets collected between 2018 and 2025, covering key-value, block, and object caches. In total, these datasets contain 5,768 traces, approximately 3.51 billion objects, and 63.75 billion requests. The details of the datasets are summarized in Table 1.

Compared policies. We compare RSM with LRU and 10 state-of-the-art cache eviction policies: heuristic policies LHD [5], ARC [31], SIEVE [54], S3-FIFO [52], and W-TinyLFU [14]; and learning-based policies LeCaR [44], CACHEUS [38], GL-Cache [49], LRB [40], and 3L-Cache [56].

Metrics. We measure object miss ratio (OMR), byte miss ratio (BMR), and throughput. Because workloads differ substantially across traces, raw miss ratios span a wide range. We therefore present the relative miss ratio $\frac{\mathrm{mr}_{\mathrm{alg}}}{\mathrm{mr}_{\mathrm{LRU}}}$, where mr denotes OMR or BMR. Since cache capacity strongly affects policy behavior, all metrics are evaluated at two representative cache sizes: 0.1% and 10% of the trace footprint (total object size), referred to as small and large cache sizes, respectively, following the setting in prior works [52, 56].

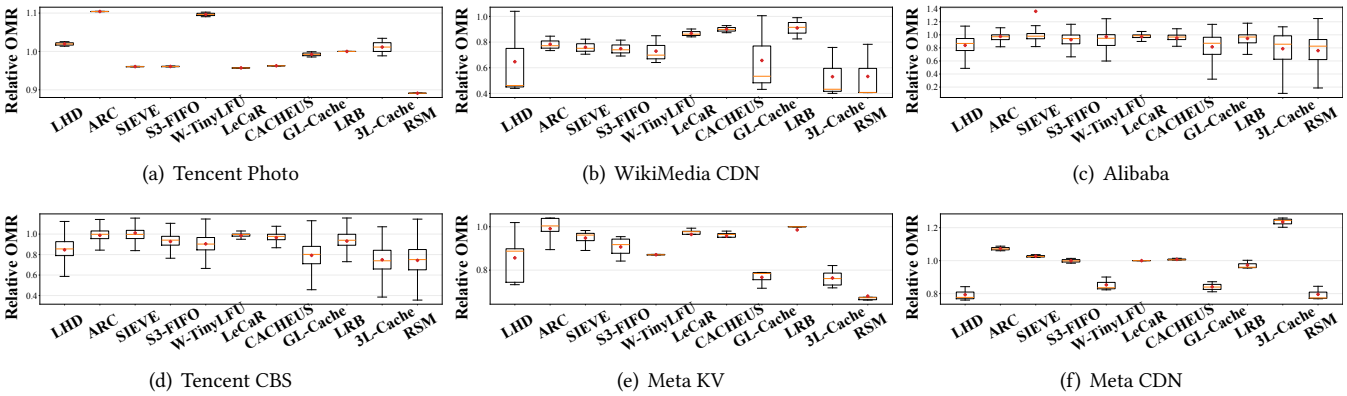
Simulator. We implement RSM in libCacheSim [1]. For all baselines, we use the parameter settings described in their original papers. In simulation, traces are replayed under stress conditions, and we record miss ratio and throughput.

Prototype. Our prototype is implemented in Cachelib [7]. It maintains the age-ordered queue with a doubly linked list, the count-indexed lists and size-range buckets with arrays and hash maps, and the score-reuse buffer with a vector. All experiments are conducted on a Supermicro X12DPi-N6 server, equipped with two Intel Xeon Gold 6348 28-core processors at 2.60 GHz and 251 GiB of RAM.

Parameter Selection. We set critical parameters through broad sensitivity sweeps and choose values near the point where the miss-ratio curve flattens. We set the middle-layer update interval to 500 requests, the feature-extreme candidate budget to 16, the random candidate budget to 256, and the score-ordered buffer size to 8. Before online optimization, we select formula terms according to the target miss ratio and the warmup one-hit ratio, defined as the fraction of objects requested exactly once when the cache first becomes full. Age and frequency are always retained. For OMR, we additionally retain s and as, since evicting a larger object leaves space for more objects. For BMR, we retain f/a in addition to the base terms. When the one-hit ratio is high, all terms are retained.

5.2 Robustness across workloads

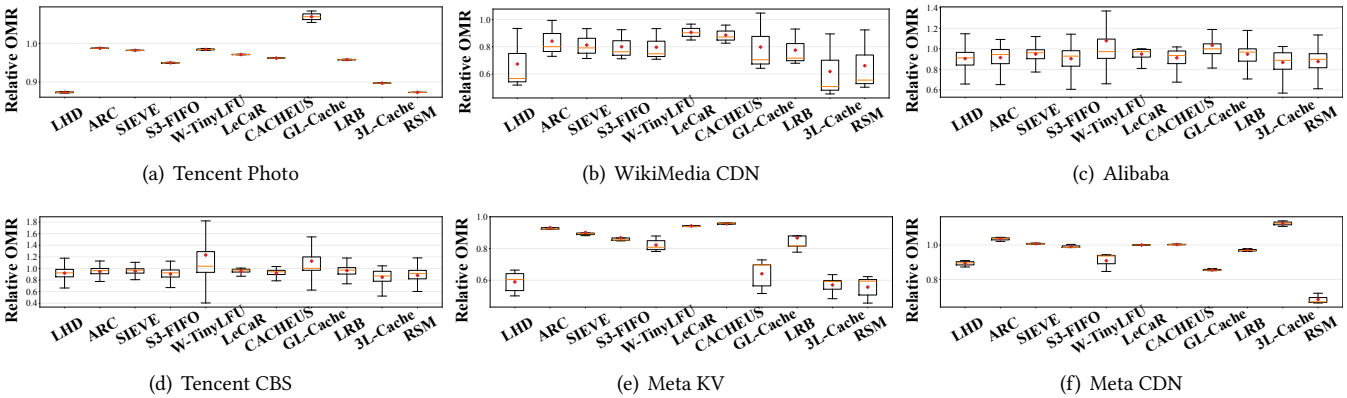
5.2.1 Object Miss Ratio. Fig. 6 and Fig. 7 show the object miss ratio for six trace groups under large and small cache sizes, respectively. RSM remains in the leading tier across all trace groups. Under the large cache size, RSM achieves the lowest OMR in Tencent Photo, Alibaba, Tencent CBS, and Meta KV, and the second-lowest OMR in WikiMedia CDN and Meta CDN. This advantage comes from the middle layer's ability to capture the appropriate balance among age, frequency, and size in each trace. Several size-aware policies also perform well in selected groups, but their performance is less consistent across workloads. For example, 3L-Cache achieves the lowest OMR in WikiMedia CDN, but it performs poorly in Tencent Photo and Meta CDN, because its time-to-next-request target does not always identify the least valuable object. LHD performs best in Meta CDN but worse in Tencent Photo, since its frequency-based design is less responsive to age shifts in that trace. GL-Cache also performs reasonably well, though it is not the best in most traces, since its group-level mechanism may evict valuable objects together with poor ones. Under the small cache size, RSM achieves the lowest OMR in Tencent Photo, Meta KV, and Meta CDN, and the second-lowest OMR in WikiMedia CDN,



(a) Tencent Photo (b) WikiMedia CDN (c) Alibaba

(d) Tencent CBS (e) Meta KV (f) Meta CDN

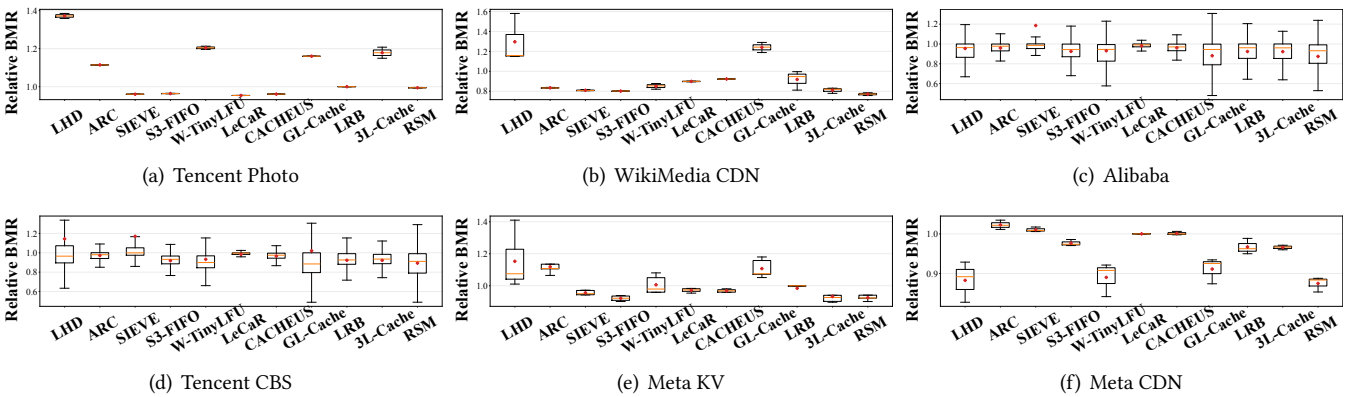
Figure 6. Relative object miss ratio across six trace groups under large cache size.



(a) Tencent Photo (b) WikiMedia CDN (c) Alibaba

(d) Tencent CBS (e) Meta KV (f) Meta CDN

Figure 7. Relative object miss ratio across six trace groups under small cache size.



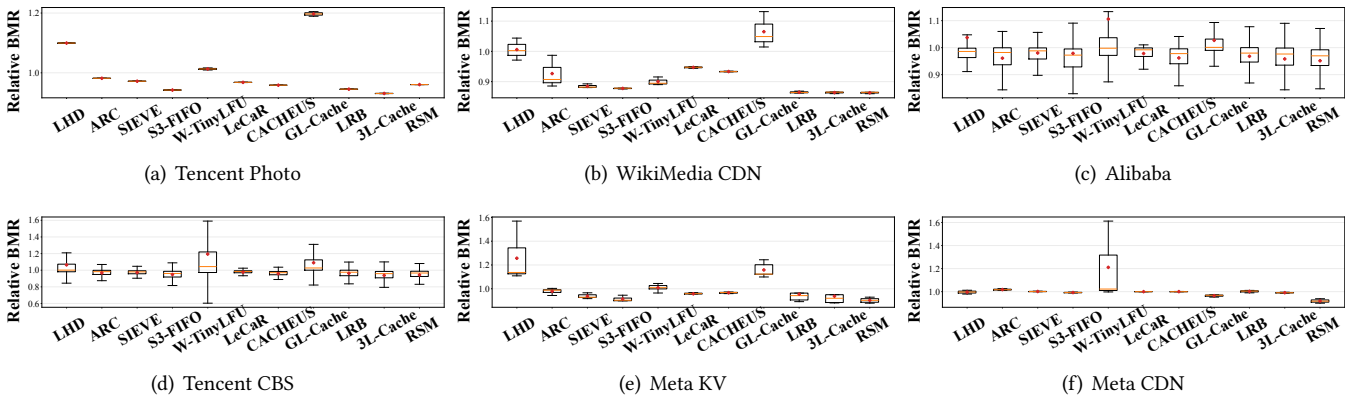
(a) Tencent Photo (b) WikiMedia CDN (c) Alibaba

(d) Tencent CBS (e) Meta KV (f) Meta CDN

Figure 8. Relative byte miss ratio across six trace groups under large cache size.

Alibaba, and Tencent CBS. 3L-Cache and LHD also show competitive OMR performance under the small cache size.

5.2.2 Byte Miss Ratio. Fig. 8 and Fig. 9 show the byte miss ratio for six trace groups under large and small cache sizes, respectively. RSM delivers consistently competitive BMR performance across all trace groups. Under the large cache size, RSM ranks first in WikiMedia CDN, Alibaba, Tencent CBS and Meta CDN. Under the small cache size, it is best in WikiMedia CDN, Alibaba, Meta CDN and Meta KV. Other policies also perform well in specific groups. For



(a) Tencent Photo (b) WikiMedia CDN (c) Alibaba

(d) Tencent CBS (e) Meta KV (f) Meta CDN

Figure 9. Relative byte miss ratio across six trace groups under small cache size.

example, under the large cache size, S3-FIFO performs best in Meta KV, while LeCaR achieves the lowest BMR in Tencent Photo. Under the small cache size, 3L-Cache performs best in Tencent Photo and Tencent CBS. However, these policies do not maintain the same level of performance across groups.

5.2.3 Overall Performance. To summarize robustness across traces, we present percentile statistics, average ranks, and robustness scores computed from miss ratios over the six trace groups.

Fig. 10 and Fig. 11 show the percentiles of OMR and BMR under both large and small cache sizes. RSM has small OMR and BMR values at lower percentiles, indicating low miss ratios on favorable trace groups, and it also has small values at upper percentiles, indicating better regression control of unfavorable cases under heterogeneous workloads. In contrast, some policies perform well at lower percentiles but degrade substantially at upper percentiles, indicating weaker robustness across traces. For example, 3L-Cache has strong lower-percentile OMR under the large cache size, but its upper-percentile OMR degrades considerably. Some policies have high miss ratios across most percentiles, indicating weak overall effectiveness. For example, ARC performs poorly across the OMR percentiles under the large cache size.

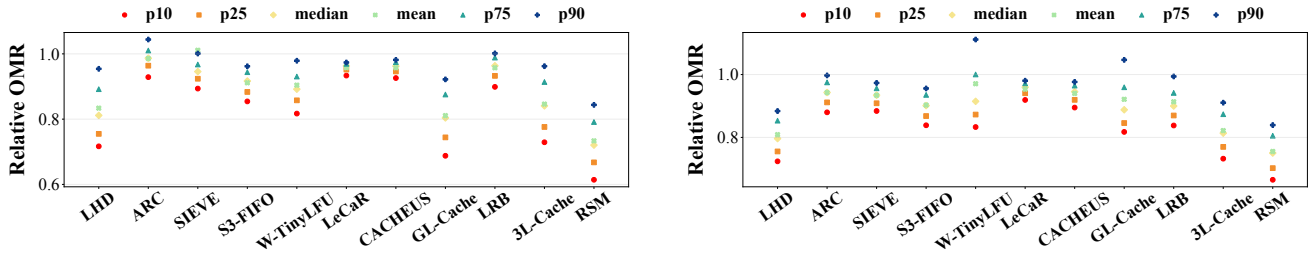
Table 2 summarizes the average rank of each policy for OMR and BMR metrics. Under both cache sizes, RSM achieves the top rank, showing that its advantage is not tied to a specific trace type. For OMR, LHD and 3L-Cache perform well under both cache sizes, whereas GL-Cache is strong only under the large cache size, since its grouping mechanism flattens differences especially when capacity is small. For byte miss ratio, S3-FIFO and 3L-Cache remain competitive under both cache sizes.

Fig. 12 shows the robustness scores of OMR and BMR across six trace groups under both large and small cache sizes. For each trace group g and policy p in the policy set $\mathcal{P}$, we first normalize the metric value using the best and worst values in that group as $s^{\text{norm}}_{g,p} = \frac{v_{g,p} - \min_{q\in\mathcal{P}} v_{g,q}}{\max_{q\in\mathcal{P}} v_{g,q} - \min_{q\in\mathcal{P}} v_{g,q}}$, and then average the normalized scores across trace groups. A lower score indicates that a policy stays closer to the best-performing policy across heterogeneous workloads. RSM achieves the best robustness score under both cache sizes and both metrics, showing that its advantage is not limited to a single workload type. By comparison, some policies score well for one metric but poorly for the other, revealing metric sensitivity. For example, LRB performs well in BMR but is much weaker in OMR, since it evicts objects based on future access distance and ignores object-size differences. LHD shows the opposite pattern. Its OMR score is favorable, but its BMR score deteriorates because it overemphasizes object size. Some policies are also sensitive to cache size. For example, GL-Cache has a competitive score under one capacity setting but loses this advantage under the other.

Table 2. Average rank across trace groups.

Policy	OMR rank		BMR rank	
	Large cache	Small cache	Large cache	Small cache
LRU	10.2	10.3	8.8	8.8
LHD	4.3	3.2	9.3	10.0
ARC	10.3	8.5	8.5	7.2
SIEVE	8.5	8.3	7.3	6.7
S3-FIFO	5.5	5.0	3.5	3.7
W-TinyLFU	6.3	7.8	6.7	10.3
LeCaR	7.7	8.7	6.8	6.8
CACHEUS	7.8	7.7	7.0	5.8
GL-Cache	3.7	7.7	7.7	9.7
LRB	7.7	6.0	5.8	5.2
3L-Cache	4.7	3.3	4.7	2.0
RSM	**1.3**	**1.5**	**1.8**	**1.8**

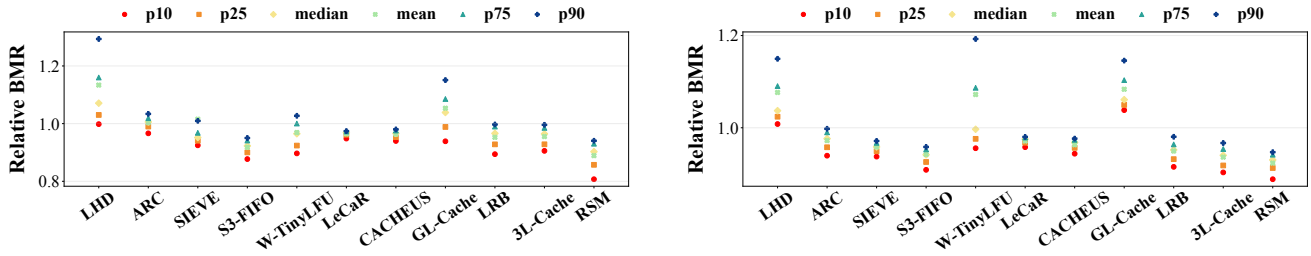
In summary, RSM's robustness across heterogeneous workloads comes from its expressiveness of workload preference in the middle layer. The scoring formula combines age, frequency, size, and interaction terms, providing a stable inductive bias for different access patterns while allowing the weights to adjust to cache feedback. As a result, RSM can



(a) OMR under large cache size

(b) OMR under small cache size

Figure 10. Percentiles of relative object miss ratio across six trace groups under two cache sizes.



(a) Large cache size

(b) Small cache size

Figure 11. Percentiles of relative byte miss ratio across six trace groups under two cache sizes.

emphasize a feature that dominates eviction decisions when such a feature exists and adjust the combination of multiple features when traces are mixed, reducing sensitivity to trace family, cache size, and evaluation metric.

5.3 Scalability

We evaluate scalability using throughput under high-pressure trace replay, as shown in Fig. 13. RSM sustains throughput comparable to heuristic policies. This is because RSM's decoupled architecture executes only the lightweight middle-layer scoring in the latency-critical path, while model optimization is performed intermittently outside the eviction path. This design keeps per-decision computation close to heuristic-level cost while preserving workload adaptability.

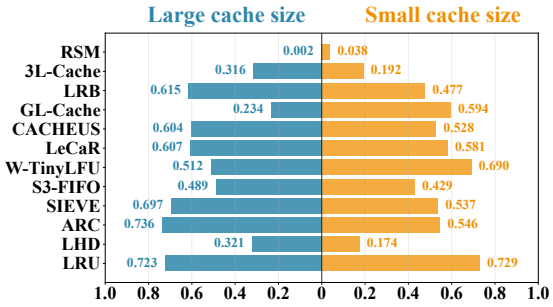
5.4 Prototype Evaluation

We implement the RSM prototype in Cachelib [7]. We verify that the prototype miss ratios are consistent with the simulator results using several randomly selected traces. Because prototype evaluation is much more time-consuming, we evaluate it using a synthetic Zipf trace, similar to prior work [15, 52]. The Zipf trace contains 100 million requests over 1 million 4 KB objects. Fig. 14 shows that RSM preserves its miss-ratio advantage while keeping throughput and CPU overhead comparable to heuristic policies. In contrast, LRB, which incurs heavier learning and inference costs, shows much lower throughput and higher CPU overhead. Throughput is calculated as the number of requests divided by wall-clock replay time. CPU time accumulates all threads, so both include asynchronous optimizer activity.

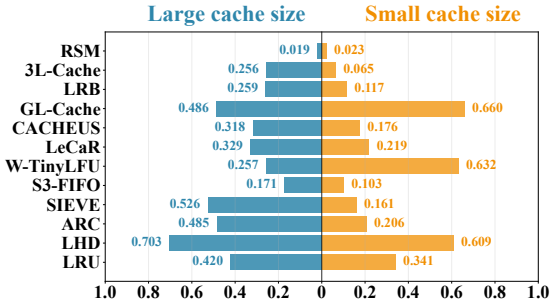
5.5 Further Analysis

We further analyze RSM's robust miss-ratio advantage.

Learning target. RSM shifts the learning target from static object utility to dynamic workload preference. As shown earlier using the example short trace in Fig. 1, a static utility, even with future knowledge, may leave a gap to the best eviction decision. We further synthesize a longer trace with a million requests similar to that short one. As Fig. 15 shows, RSM can achieve better OMR and BMR than Belady and BeladySize in this case, since it learns workload eviction preferences to characterize how effective eviction decisions are made in the current workload, rather than learning a static object utility. We also compare learned workload preferences with the fixed feature combinations of GDSF and LRFU (not shown in Fig. 15). With results normalized to LRU and averaged equally across the six workload families, at 10% capacity RSM reduces OMR/BMR by 17.29%/6.44%



(a) Object miss ratio



(b) Byte miss ratio

Figure 12. Robustness scores under two cache sizes.

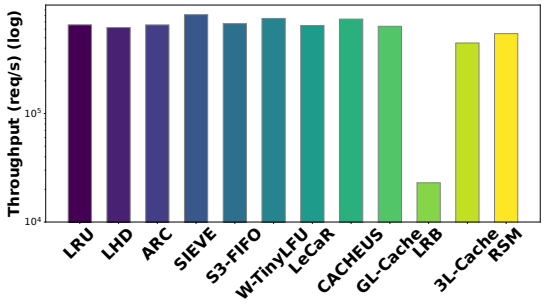
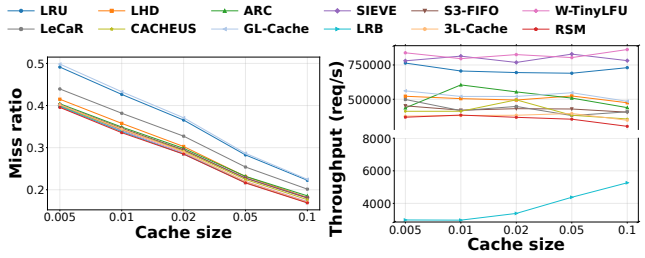


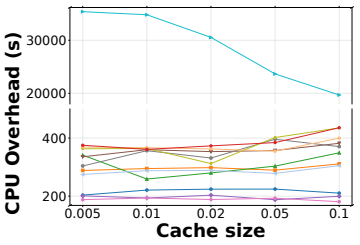
Figure 13. Throughput averaged over two cache sizes.

versus GDSF [12] and 26.79%/11.70% versus LRFU [24]. At 0.1%, the reductions are 13.84%/4.64% and 29.50%/9.83%, respectively. GDSF and LRFU use fixed scoring tradeoffs across workloads, whereas RSM learns workload-specific balances among recency, frequency, size, and interactions.

Selected features. We replace RSM's features with features adopted by other algorithms. 3L-Cache uses age, size, frequency and the three most recent inter-arrival times. LRB uses size, 32 recent inter-arrival times, 10 exponentially decayed counters. Fig. 16 shows that larger alternative feature sets perform worse because they disperse learning and slow convergence, although weights can still stabilize. To test an uninformative characteristic, we add a fresh-random characteristic regenerated independently at every eviction. Its final-window weight is at most 0.01 in 98% of runs, with median 0.0011. Thus, RSM can suppress an uninformative characteristic rather than retaining material weight on every supplied input.



(a) Miss ratio (b) Throughput



(c) CPU overhead

Figure 14. Cachelib prototype results.

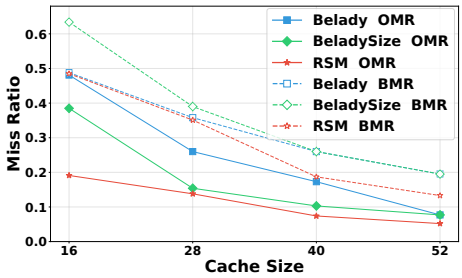


Figure 15. Performance comparison between object utility and workload preference.

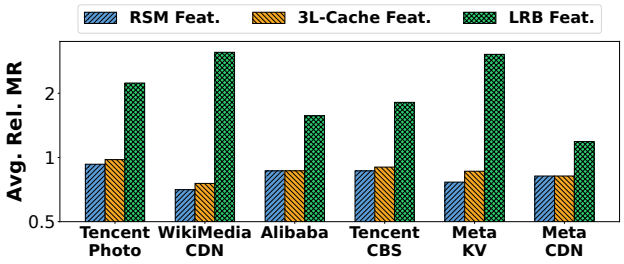


Figure 16. Average relative miss ratio of different features.

Learning model. We compare CMA-ES with DRL and GBM that is used by many algorithms such as LRB and 3L-Cache. Fig. 17 shows that CMA-ES outperforms DRL and GBM for learning the middle-layer weights. One possible reason is that DRL learns more slowly and is more prone to local optima, and GBM is less suited to this label-free optimization task.

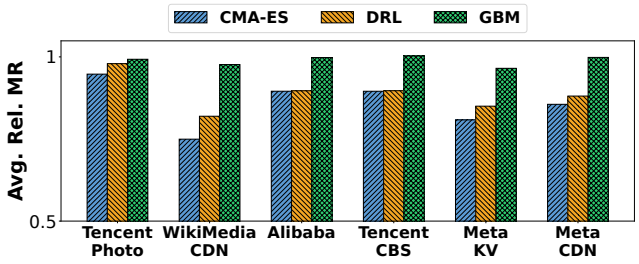


Figure 17. Average relative miss ratio of different models.

Adaptation Effectiveness. We classify workloads by five request-stream characteristics: object-size variability, access concentration, reuse prevalence, short-term reuse, and hot-set turnover. For each, lower and upper quartiles form low and high classes, and we compare the corresponding learned size, frequency, or recency weight. The weights differ clearly between classes. For example, the high-reuse class learns a frequency weight 52.4% higher than the low-reuse class, and the between-class frequency-weight distance is 3.40× the within-class distance. This agrees with repeated accesses making frequency more informative for retention. Thus, weights reflect workload characteristics rather than remaining uniform across traces.

Adaptation speed. We evaluate two settings. Controlled workloads switch between phases where recency and frequency are respectively more useful, giving a known preference change. Real-trace workloads concatenate production-trace segments with different access patterns, testing adaptation without synthetic regularity. Adaptation is measured from the boundary until five consecutive weight vectors are closer to weights learned for the new workload. Median adaptation is 106.05k requests for controlled shifts and 113.5k for real-trace shifts.

6 Related Work

6.1 Heuristic-based Eviction Policies

Heuristic policies rely on fixed rules to evict objects [2, 3, 5, 10, 12–14, 21–25, 31, 33, 35–37, 42, 51]. Traditional heuristics often depend on a single observable feature. For example, LRU, LFU, LSU, and FIFO policies prioritize eviction based on recency, frequency, size, and arrival order, respectively. Building on this, various composite heuristics attempt to improve performance by combining multiple features. ARC [31] dynamically balances between recency and frequency by adjusting the relative capacity of its LRU and LFU queues. W-TinyLFU [14] adopts a space-efficient frequency sketch to filter one-time accesses before they enter the main cache space. Furthermore, sophisticated heuristics define derived metrics to capture complex workload patterns such as LHD [5], which defines the expected cache hit count per unit size as the hit density metric. Recently, hardware-efficient heuristics have regained attention through methods like SIEVE [54] and S3-FIFO [52], which deploy quick demotion and lazy promotion rules on simple FIFO structures.

6.2 Learning-based Eviction Policies

Learning-based schemes have been proposed to improve adaptability beyond fixed heuristics [8, 11, 16–19, 27–30, 34, 38–41, 43–45, 47–49]. They commonly learn object-level utilities. LRB [40] trains Gradient Boosting Machines (GBM) to predict reuse distance online and evicts the object with the largest predicted reuse distance. Raven [17] builds a density estimation network based on Gaussian mixture models to estimate the reuse distance distribution of individual objects. Deep reinforcement learning has also been explored, such as LRU-BaSE [45], which applies a DRL agent to choose eviction victims from the tail portion of an LRU list. Some recent designs reduce inference latency by coarsening or amortizing learned decisions, but this can sacrifice eviction accuracy. GL-Cache [49] clusters objects by recent access time and uses GBM models to train and predict utilization at the group level. 3L-Cache [56] lowers training frequency and reuses the same prediction results across many subsequent eviction decisions. Instead of directly learning object scores, LeCaR [44] and CACHEUS [38] use online learning with regret minimization to control the dynamic use of two heuristic cache replacement policies.

7 Conclusion

This paper presents RSM, a cache eviction framework that achieves robust cache-hit performance and scalable request processing through a workload-preference middle layer. RSM moves beyond static object-utility learning by reorienting what to learn and how to use the learned model during eviction. It learns how the current workload prioritizes different eviction-relevant features and encodes these preferences as learned feature weights in a compact parameterized scoring formula over a concise set of representative features at the middle layer. The learning model is decoupled from the latency-critical eviction path and updates the middle layer intermittently to reflect the latest workload preferences, preserving eviction quality while improving scalability. Evaluations over 5,768 real-world traces show that RSM achieves robust object and byte miss ratios across traces under both cache sizes while maintaining throughput comparable to heuristic policies. The Cachelib prototype further confirms the practicality of RSM in production caches.

Acknowledgments

We thank our shepherd Phil Shilane and anonymous reviewers for their insightful feedback. This work is supported in part by the National Natural Science Foundation of China under Grants No. 62302472, No. 62372425, and No. 62572450.

References

[1] 2026. libCacheSim. Retrieved October 2026 from https://github.com/1a1a11a/libCacheSim/

[2] Bahman Abolhassani, John Tadrous, and Atilla Eryilmaz. 2021. Single vs distributed edge caching for dynamic content. *IEEE/ACM Transactions on Networking* 30, 2 (2021), 669–682.

[3] Bahman Abolhassani, John Tadrous, Atilla Eryilmaz, and Edmund Yeh. 2021. Fresh caching for dynamic content. In *Proceedings of the 40th IEEE International Conference on Computer Communications (INFOCOM)*. IEEE, 1–10.

[4] Anne Auger and Nikolaus Hansen. 2005. A restart CMA evolution strategy with increasing population size. In *Proceedings of the 2005 IEEE congress on evolutionary computation*, Vol. 2. IEEE, 1769–1776.

[5] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. 2018. LHD: Improving cache hit rate by maximizing hit density. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 389–403.

[6] Laszlo A. Belady. 1966. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal* 5, 2 (1966), 78–101.

[7] Benjamin Berg, Daniel S. Berger, Sara Hunter, Mike Kaminsky, Michael Kozuch, Justine Sherry, Padmanabhan Pillai, and Gregory R. Ganger. 2020. The CacheLib caching engine: Design and experiences at scale. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 753–768.

[8] Daniel S. Berger. 2018. Towards lightweight and robust machine learning for CDN caching. In *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (HotNets)*. ACM, 134–140.

[9] Daniel S. Berger, Nathan Beckmann, and Mor Harchol-Balter. 2018. Practical Bounds on Optimal Caching with Variable Object Sizes. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 2, 2 (2018), 1–38.

[10] Aaron Blankstein, Siddhartha Sen, and Michael J. Freedman. 2017. Hyperbolic caching: Flexible caching for web applications. In *Proceedings of the 2017 USENIX Annual Technical Conference (ATC)*. 499–511.

[11] Zheng Chang, Lei Lei, Zhenyu Zhou, Shiwen Mao, and Tapani Ristaniemi. 2018. Learn to cache: Machine learning for network edge caching in the big data era. *IEEE Wireless Communications* 25, 3 (2018), 28–35.

[12] Ludmila Cherkasova. 1998. *Improving WWW proxies performance with greedy-dual-size-frequency caching policy.* Hewlett-Packard Laboratories Palo Alto, CA, USA.

[13] Ludmila Cherkasova and Gianfranco Ciardo. 2001. Role of aging, frequency, and size in web cache replacement policies. In *Proceedings of the 9th International Conference on High-Performance Computing and Networking (HPCN)*. Springer, 114–123.

[14] Gil Einziger, Roy Friedman, and Ben Manes. 2017. TinyLFU: A highly efficient cache admission policy. *ACM Transactions on Storage* 13, 4 (2017), 1–31.

[15] Bin Fan, David G. Andersen, and Michael Kaminsky. 2013. MemC3: Compact and concurrent MemCache with dumber caching and smarter hashing. In *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 371–384.

[16] Qilin Fan, Xiuhua Li, Jian Li, Qiang He, Kai Wang, and Junhao Wen. 2021. PA-Cache: Evolving learning-based popularity-aware content caching in edge networks. *IEEE Transactions on Network and Service Management* 18, 2 (2021), 1746–1757.

[17] Xinyue Hu, Eman Ramadan, Wei Ye, Feng Tian, and Zhi-Li Zhang. 2022. Raven: Belady-guided, predictive (deep) learning for in-memory and content caching. In *Proceedings of the 18th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT)*. ACM, 72–90.

[18] Akanksha Jain and Calvin Lin. 2016. Back to the future: Leveraging Belady's algorithm for improved cache replacement. In *Proceedings of the 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 78–89.

[19] Akanksha Jain and Calvin Lin. 2018. Rethinking Belady's algorithm to accommodate prefetching. In *Proceedings of the 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 110–123.

[20] Grahame A Jastrebski and Dirk V Arnold. 2006. Improving evolution strategies through active covariance matrix adaptation. In *Proceedings of the 2006 IEEE international conference on evolutionary computation*. IEEE, 2814–2821.

[21] Song Jiang and Xiaodong Zhang. 2002. LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. *ACM SIGMETRICS Performance Evaluation Review* 30, 1 (2002), 31–42.

[22] Theodore Johnson and Dennis Shasha. 1994. 2Q: A low overhead high performance buffer management replacement algorithm. In *Proceedings of the 20th VLDB Conference (VLDB)*. 439–450.

[23] Donghee Lee, Jongmoo Choi, Jong-Hun Kim, Sam H. Noh, Sang Lyul Min, Yookun Cho, and Chong Sang Kim. 1999. On the existence of a spectrum of policies that subsumes the least recently used (LRU) and least frequently used (LFU) policies. In *Proceedings of the 1999 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS)*. ACM, 134–143.

[24] Donghee Lee, Jongmoo Choi, Jong-Hun Kim, Sam H Noh, Sang Lyul Min, Yookun Cho, and Chong Sang Kim. 2001. LRFU: A spectrum of policies that subsumes the least recently used and least frequently used policies. *IEEE transactions on Computers* 50, 12 (2001), 1352–1361.

[25] Cong Li. 2018. DLIRS: Improving low inter-reference recency set cache replacement policy with dynamics. In *Proceedings of the 11th ACM International Systems and Storage Conference (SYSTOR)*. ACM, 59–64.

[26] Jinhong Li, Qiuping Wang, Patrick P. C. Lee, and Chao Shi. 2020. An in-depth analysis of cloud block storage workloads in large-scale production. In *Proceedings of the 2020 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 37–47.

[27] Suoheng Li, Jie Xu, Mihaela van der Schaar, and Weiping Li. 2016. Trend-aware video caching through online learning. *IEEE Transactions on Multimedia* 18, 12 (2016), 2503–2516.

[28] Evan Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. 2020. An imitation learning approach for cache replacement. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*. PMLR, 6237–6247.

[29] Ke Liu, Kan Wu, Hua Wang, Ke Zhou, Peng Wang, Ji Zhang, and Cong Li. 2024. SLAP: Segmented reuse-time-label based admission policy for content delivery network caching. *ACM Transactions on Architecture and Code Optimization* 21, 2 (2024), 1–24.

[30] Thodoris Lykouris and Sergei Vassilvitskii. 2021. Competitive caching with machine learned advice. *J. ACM* 68, 4 (2021), 1–25.

[31] Nimrod Megiddo and Dharmendra S Modha. 2003. ARC: A Self-Tuning, low overhead replacement cache. In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST)*.

[32] Meta. 2026. Evaluating SSD Hardware for Facebook Workloads. Retrieved October 2026 from https://cachelib.org/docs/Cache_Library_User_Guides/Cachebench_FB_HW_eval/

[33] Elizabeth J. O'Neil, Patrick E. O'Neil, and Gerhard Weikum. 1993. The LRU-K page replacement algorithm for database disk buffering. *ACM SIGMOD Record* 22, 2 (1993), 297–306.

[34] Nitish K. Panigrahy, Philippe Nain, Giovanni Neglia, and Don Towsley. 2021. A new upper bound on cache hit probability for non-anticipative caching policies. *ACM SIGMETRICS Performance Evaluation Review* 48, 3 (2021), 138–143.

[35] Sejin Park and Chanik Park. 2017. FRD: A filtering based buffer cache algorithm that considers both frequency and reuse distance. In *Proceedings of the 33rd IEEE International Conference on Massive Storage Systems and Technology (MSST)*.

[36] Seon-yeong Park, Dawoon Jung, Jeong-uk Kang, Jin-soo Kim, and Joonwon Lee. 2006. CFLRU: A replacement algorithm for flash memory. In *Proceedings of the 2006 International Conference on Compilers, Architecture and Synthesis for Embedded Systems (CASES)*. ACM, 234–241.

[37] Luigi Rizzo and Lorenzo Vicisano. 2000. Replacement policies for a proxy cache. *IEEE/ACM Transactions on Networking* 8, 2 (2000), 158–170.

[38] Liana V. Rodriguez, Farzana Yusuf, Steven Lyons, Eysler Paz, Raju Rangaswami, Jason Liu, Ming Zhao, and Giri Narasimhan. 2021. Learning cache replacement with CACHEUS. In *Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST)*. 341–354.

[39] Alireza Sadeghi, Fatemeh Sheikholeslami, and Georgios B. Giannakis. 2017. Optimal and scalable caching for 5G using reinforcement learning of space-time popularities. *IEEE Journal of Selected Topics in Signal Processing* 12, 1 (2017), 180–190.

[40] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. 2020. Learning relaxed Belady for content distribution network caching. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 529–544.

[41] Zhenyu Song, Kevin Chen, Nikhil Sarda, Deniz Altınbüken, Eugene Brevdo, Jimmy Coleman, Xiao Ju, Pawel Jurczyk, Richard Schooler, and Ramki Gummadi. 2023. HALP: Heuristic aided learned preference eviction policy for YouTube content delivery network. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 1149–1163.

[42] Bijeta Subedi. 2012. *An evaluation of page replacement algorithm based on low inter-reference recency set (LIRS) scheme on weak locality workloads*. Ph. D. Dissertation. Department of Computer Science and Information Technology.

[43] Linpeng Tang, Qi Huang, Amit Puntambekar, Ymir Vigfusson, Wyatt Lloyd, and Kai Li. 2017. Popularity prediction of Facebook videos for higher quality streaming. In *Proceedings of the 2017 USENIX Annual Technical Conference (ATC)*. 111–123.

[44] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. 2018. Driving cache replacement with ML-based LeCaR. In *Proceedings of the 10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*.

[45] Peng Wang, Hong Jiang, Yu Liu, Zhelong Zhao, Ke Zhou, and Zhihai Huang. 2024. Beyond Belady to attain a seemingly unattainable byte miss ratio for content delivery networks. *IEEE Transactions on Parallel and Distributed Systems* 35, 11 (2024), 1949–1963.

[46] Wikimedia Foundation. 2025. Data Platform/Data Lake/Traffic/Caching. Retrieved October 2026 from https://wikitech.wikimedia.org/wiki/Data_Platform/Data_Lake/Traffic/Caching

[47] Gang Yan and Jian Li. 2020. RL-Belady: A unified learning framework for content caching. In *Proceedings of the 28th ACM International Conference on Multimedia (MM)*. ACM, 1009–1017.

[48] Gang Yan, Jian Li, and Don Towsley. 2021. Learning from optimal caching for content delivery. In *Proceedings of the 17th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT)*. ACM, 344–358.

[49] Juncheng Yang, Ziming Mao, Yao Yue, and K. V. Rashmi. 2023. GL-cache: Group-level learning for efficient and high-performance caching. In *Proceedings of the 21st USENIX Conference on File and Storage Technologies (FAST)*. 115–134.

[50] Juncheng Yang, Yao Yue, and Rashmi Vinayak. 2021. A large-scale analysis of hundreds of in-memory key-value cache clusters at Twitter. *ACM Transactions on Storage* 17, 3 (2021), 1–35.

[51] Juncheng Yang, Yao Yue, and Rashmi Vinayak. 2021. Segcache: A memory-efficient and scalable in-memory key-value cache for small objects. In *Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 503–518.

[52] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and Rashmi Vinayak. 2023. FIFO queues are all you need for cache eviction. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. ACM, 130–149.

[53] Yu Zhang, Ping Huang, Ke Zhou, Hua Wang, Jianying Hu, Yongguang Ji, and Bin Cheng. 2020. OSCA: An Online-Model based cache allocation scheme in cloud block storage systems. In *Proceedings of the 2020 USENIX Annual Technical Conference (ATC)*. USENIX Association, 785–798.

[54] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K. V. Rashmi. 2024. SIEVE is simpler than LRU: An efficient turn-key eviction algorithm for web caches. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 1229–1246.

[55] Ke Zhou, Si Sun, Hua Wang, Ping Huang, Xubin He, Rui Lan, Wenyan Li, Wenjie Liu, and Tianming Yang. 2018. Demystifying cache policies for photo stores at scale: A Tencent case study. In *Proceedings of the 32nd ACM International Conference on Supercomputing (ICS)*. ACM, 284–294.

[56] Wenbin Zhou, Zhixiong Niu, Yongqiang Xiong, Juan Fang, and Qian Wang. 2025. 3L-cache: Low overhead and precise learning-based eviction policy for caches. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*. 237–254.